

1. Environment Setup, Tests, Maintenance, and End-to-End Tutorial

This document combines repository setup, Python environment checks, definition-table locations, regression tests, round-trip artifact generation, maintenance, and the end-to-end tutorial.

For the conversion model, see [02_STRUCTURED_CSV_LHM_BINDINGS.md](#). For implementation details, use the phase-specific documents.

Part A. Environment, Definitions, Tests, and Maintenance

1. Purpose

This document explains how to prepare and verify the GitHub workspace used by the UADC PoC. It identifies the editable and generated EN 16931, UBL, ADS, and ISO 21378 ADC definition tables, the required Python environment, the test programs, and the committed output evidence.

The processing guides are separate:

- **01_ENVIRONMENT_TESTS_TUTORIAL.md** provides a short end-to-end walkthrough.
- **03_PHASE1_UBL_SYNTAX_BINDING.md** specifies Phase 1.
- **04_PHASE2_ADS_PSV_SEMANTIC_BINDING.md** specifies Phase 2.
- **02_STRUCTURED_CSV_LHM_BINDINGS.md** specifies the common model, taxonomy, and supporting tools.

2. Repository Layout

Directory	Role
src/	Operational Phase 1 and Phase 2 converters.
src/tutorial/	Beginner wrappers that call the operational converters.
tools/	Definition maintenance, taxonomy generation, artifact generation, and inspection tools.
tools/tutorial/	Simplified converter implementations for learning and experiments.
tests/	Directly executable regression scripts and helpers.
specs/	LHM, binding, currency, and XBRL GL definition tables.
samples/	Committed input XML and expected sample data.
out/	Generated taxonomy, Phase 1, Phase 2, reverse, tutorial, and QA evidence tracked by Git.
docs/	The five canonical guides and decision records.

Each script directory contains a short **README.md** for GitHub browsing. The five documents in **docs/** are the authoritative operating and implementation guides.

3. Clone and Python Setup

3.1 Windows PowerShell

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd .\UADC-PoC
$python = 'python'
& $python --version
```

If Python is not on **PATH**, use the installed executable path:

```
$python = 'C:\Users\  
<user>\AppData\Local\Programs\Python\Python310\python.exe'  
& $python --version
```

3.2 macOS or Linux

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd ./UADC-PoC
PYTHON=python3
$PYTHON --version
```

3.3 Core Compilation Check

```
& $python -m py_compile `  
.\src\syntax_binding.py `  
.\src\semantic_binding.py `  
.\src\syntax_binding_ads_xbrl_gl.py
```

All Python programs under **tools/** can be checked with:

```
Get-ChildItem .\tools -Recurse -Filter *.py | ForEach-Object {  
    & $python -m py_compile $_.FullName  
}
```

4. Definition Tables

4.1 EN 16931 LHM

File	Purpose
specs/lhm/source/EN16931_CIOUS_Invoice_LHM_Source.csv	Editable source containing business terms and override columns.
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv	Generated operational LHM consumed by converters and taxonomy generation.
specs/lhm/EN16931_CIOUS_Invoice.xlsx	Local reviewer workbook; not the runtime authority.

The LHM records semantic hierarchy, class and attribute type, multiplicity, effective **lhm_level**, semantic path, element name, datatype, and UBL XPath.

4.2 Phase 1 UBL Syntax Binding

The operational UBL Invoice binding is:

```
specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv
```

It contains both class rows and fact rows. It is the runtime authority for XML locations, selectors, Structured CSV columns, and repeated row scopes.

4.3 Phase 2 ADS XBRL GL Bindings

Six syntax-binding CSVs under **specs/bindings/syntax/** define:

- Invoices Received;
- Invoices Generated;
- Invoices Received Lines;
- Invoices Generated Lines;
- Supplier Listing;
- Customer Master.

The review workbook is:

```
specs/bindings/ADS_XBRL_GL_Bindings.xlsx
```

4.4 Phase 2 ADS PSV Bindings

Six semantic-binding CSVs under **specs/bindings/semantic/** define the same header, line, supplier, and customer target families as delimiter-separated files.

4.5 ISO 21378 ADC Bindings

The four ISO 21378:2019 invoice views are:

```
ISO21378_SAL_Invoice_Generated_CSV_Binding.csv
ISO21378_SAL_Invoice_Generated_Details_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_Details_CSV_Binding.csv
```

They represent Tables 38, 39, 53, and 54. Mapping status and notes distinguish direct values, approximations, transformations, and ERP information unavailable from an EN 16931 invoice.

4.6 Reference Tables

- **specs/Currency.csv** maps ISO 4217 currency codes to minor units.
- **specs/CountryCurrency.csv** provides country/currency example data.
- **specs/XBRL-GL/** contains XBRL GL definition references used for bindings.

5. Samples and Generated Evidence

5.1 Phase 1 Inputs

```
samples/input/openpeppol_ubl_invoice_minimal.xml
samples/input/bis-billing3-examples/*.xml
```

The current regression set contains one minimal invoice and nine BIS Billing 3 examples.

5.2 Output Layout

```
out/taxonomy/
out/phase1/
out/phase2/ADS_XBRL_GL/
out/phase2/ADS_PSV/
out/phase2/ISO21378_ADC/
out/reverse/
out/tutorial/
```

Generated evidence is committed for review. Regenerate it from definitions and scripts instead of editing it manually.

6. Environment Verification

Run the tutorial prerequisite check:

```
& $python .\src\tutorial\00_check_environment.py
```

It verifies the three operational scripts, the taxonomy generator, LHM, UBL and ADS bindings, and the minimal sample. A missing generated taxonomy is reported as a next action rather than a missing source definition.

Generate and validate the local taxonomy when needed:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
& $python .\tests\validate_taxonomy.py
```

Optional dependencies:

Dependency	Purpose
Arelle	xBRL-CSV metadata and XBRL taxonomy or instance validation.
xmlschema	UBL 2.1 round-trip schema validation.
pypdf	EN 16931 coverage audit against a PDF.
pdfplumber	Updating LHM definitions from EN 16931 Table 2.

7. Regression Tests

Tests are plain Python scripts and can be executed directly.

7.1 Phase 1

```
& $python .\tests\test_lhm_semantic_paths.py
& $python .\tests\test_lhm_hierarchical_csv_layout.py
& $python .\tests\test_syntax_binding.py
& $python .\tests\test_syntax_binding_reverse.py
& $python .\tests\test_bis_billing3_examples_conversion.py
& $python .\tests\test_roundtrip_artifacts.py
& $python .\tests\test_roundtrip_xml_ubl_schema.py
& $python .\tests\test_xbrl_csv_metadata_arelle.py
```

7.2 Phase 2 ADS XBRL GL

Run all target-specific scripts named **test_ads*_xbrl_gl.py**, followed by:

```
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

7.3 Phase 2 Semantic Outputs

```
& $python .\tests\test_ads_invoices_received_psv.py
& $python .\tests\test_ads_invoices_generated_psv.py
& $python .\tests\test_semantic_binding_csv_format.py
& $python .\tests\test_semantic_binding_indexed_paths.py
```

7.4 Full Direct Run

```
Get-ChildItem .\tests\test_*.py | Sort-Object Name | ForEach-Object {
    & $python $_.FullName
    if ($LASTEXITCODE -ne 0) { throw "Failed: $($_.Name)" }
}
```

The latest recorded run completed 22 scripts with no failures.

8. Test Internals

Tests create subprocess commands with the current Python executable so the converter and test use the same environment. **phase1_helpers.py** centralizes the Phase 1 binding path and conversion helper used by target tests.

The round-trip builder recreates four artifact folders for every case: source XML, Structured CSV, metadata JSON, and regenerated UBL XML. The schema test checks XML validity rather than byte-for-byte identity. The Arelle test checks the metadata JSON and its taxonomy references.

Target tests assert semantic facts, hierarchy, selectors, row scopes, XBRL GL tuple placement, monetary decimals, and output naming. They are regression contracts, not only smoke tests.

9. GitHub Workflow

Before work:

```
git status --short
git pull --ff-only
```

After changes:

1. Run relevant focused tests.

2. Run the complete regression set for shared model or converter changes.
3. Regenerate committed output evidence.
4. Regenerate changed documentation PDFs.
5. Review **git diff** and **git status**.
6. Commit and push to the intended branch.

The binding CSV and source LHM definitions are reviewed source files. Generated CSV, JSON, XML, XSD, linkbase, and PDF evidence must remain reproducible from those files and the committed scripts.

10. Documentation PDF Workflow

Markdown is the editable source. PDFs are generated with the VSCode Markdown PDF extension using the configured margins:

- top and bottom: **20mm**;
- left and right: **18mm**.

After export, inspect the first and last pages and any pages containing large tables or code blocks. Commit the Markdown and corresponding PDF together.

10.1 Regenerating Japanese Documentation

Japanese Markdown is generated below a **ja/** subdirectory in every directory that contains first-party project documentation. The English Markdown remains the structural source. Project terminology and approved Japanese expressions are defined in **docs/ja/TERMINOLOGY.csv**.

The terminology CSV uses these columns:

Column	Purpose
source_term	English term detected in the source Markdown.
ja_term	Japanese expression inserted into generated documents.
definition_ja	Japanese definition used to review the intended meaning.
match_mode	literal for case-sensitive identifiers or phrase for natural-language matching.
notes	Editorial guidance, abbreviations, or plural-form notes.

Edit the CSV in Excel and save it as **CSV UTF-8 (comma delimited)**. Confirm that the saved file retains the UTF-8 BOM. The generator reads it with **utf-8-sig**, so the BOM is accepted and excluded from the first column name.

From the repository root, regenerate all Japanese Markdown:

```
$python =
'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe'
```

```
& $python .\tools\translate_markdown_ja.py
```

The local Argos Translate runtime and English-to-Japanese model must already exist under **.translation_runtime/** and **.translation_data/**. These large local dependencies are ignored by Git. The generator sends no document text to an external translation service.

Validate the generated set without rewriting it:

```
& $python .\tools\translate_markdown_ja.py --check
```

The check confirms that every supported English Markdown file has a Japanese counterpart, that code-fence counts match, and that every output contains Japanese text. Before publication, also review terminology, relative links, tables, commands, paths, XPath, and the absence of unreplaced protection markers. A correction to **ja_term** is applied to every generated Japanese document on the next full run.

11. Troubleshooting

11.1 Python is not found

Set **\$python** or **PYTHON** to an absolute executable path.

11.2 PDF tools cannot import a dependency

Install only the required package in the selected Python environment, or skip the optional PDF-derived maintenance action.

11.3 Arelle or UBL schema validation is unavailable

Run the remaining tests and report the omitted external validation explicitly.

11.4 Generated output differs unexpectedly

Check the active binding path, input stem, encoding, taxonomy version, currency table, and whether the output was regenerated with the operational script under **src/** rather than a simplified tutorial tool.

12. Development Environment and Maintenance

Development And Tooling Guide

This guide explains clone-time setup, local development checks, and model-maintenance tools.

Directory Roles


```
src/      operational conversion scripts
tools/    setup, model maintenance, taxonomy generation, and test artifact
helpers
tests/    regression checks
specs/    committed LHM, binding, currency, and model definition CSVs
samples/  committed sample XML and expected outputs
out/      generated PoC evidence and target output tracked by Git
```

The main invoice conversion runtime is in **src/**. Use **tools/** for setup, regeneration, test artifact building, taxonomy output, and specification maintenance.

Initial Setup After Clone

Windows PowerShell:

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd .\UADC-PoC
$python = 'python'
& $python --version
```

\$python = 'python' is a PowerShell variable assignment used by examples. It means run the Python executable available on **PATH**. If Python is not on **PATH**, set it to the full executable path:

```
$python = 'C:\Users\  
<user>\AppData\Local\Programs\Python\Python310\python.exe'  
& $python --version
```

macOS or Linux shell:

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd ./UADC-PoC
PYTHON=python3
$PYTHON --version
```

Use **\$PYTHON script.py** on macOS/Linux. Use **& \$python script.py** only in Windows PowerShell.

Compile Core Scripts

Compile before running broad checks:

```
& $python -m py_compile `
.\src\syntax_binding.py `
.\src\syntax_binding_ads_xbrl_gl.py `
.\src\semantic_binding.py `
.\tools\build_roundtrip_test_artifacts.py `
.\tools\taxonomy\XBRLGL_TaxonomyGenerator.py
```

macOS/Linux equivalent:

```
$PYTHON -m py_compile \
./src/syntax_binding.py \
./src/syntax_binding_ads_xbrl_gl.py \
./src/semantic_binding.py \
./tools/build_roundtrip_test_artifacts.py \
./tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

Tool Classification

GitHub and development environment support:

```
build_roundtrip_test_artifacts.py
psv_viewer.html
```

Conversion-target model environment support:

```
taxonomy/xBRLGL_TaxonomyGenerator.py
build_lhm_from_source.py
build_syntax_binding.py
normalize_lhm_semantic_paths.py
normalize_lhm_class_element.py
check_lhm_class_element.py
audit_en16931_coverage.py
extend_en16931_lhm_coverage.py
order_lhm_by_en16931_table2.py
update_lhm_definitions_from_pdf.py
update_lhm_syntax_sequence_from_ubl_xsd.py
```

Tutorial and sample converters:

```
tools/tutorial/syntax_binding_sample.py  
tools/tutorial/semantic_binding_sample.py
```

Operational runtime converters are in **src/**, not **tools/**.

Model Inputs And Generated Outputs

Committed source definitions:

```
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv  
specs/bindings/syntax/  
specs/bindings/semantic/  
specs/Currency.csv
```

Generated local outputs:

```
out/taxonomy/  
out/phase1/  
out/phase2/  
tests/roundtrip/
```

out/ contains generated evidence and target outputs. It is included in Git for the current PoC so Phase 1 and Phase 2 results remain reviewable through GitHub. Regenerate committed outputs from their source definitions and scripts rather than editing them manually.

Detailed purpose, input/output, command-line, processing, function, validation, dependency, and side-effect specifications for all 15 programs under **tools/** are documented in

02_STRUCTURED_CSV_LHM_BINDINGS.md.

Taxonomy Generation

Primary script:

```
tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

Role:

```
LHM CSV -> local xBRL-CSV taxonomy for Structured CSV metadata
```

Generate and validate through regression checks:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py  
& $python .\tests\validate_taxonomy.py
```

Expected local output includes:

```
out/taxonomy/en16931/en16931-2026-07-05.xsd  
out/taxonomy/gen/gl-gen-2026-07-05.xsd  
out/taxonomy/plt/en16931-oim-2026-07-05.xsd  
out/taxonomy/plt/en16931-def-2026-07-05.xml
```

src/syntax_binding.py does not generate taxonomy files. It references them through **--taxonomy-base** when writing xBRL-CSV metadata JSON.

LHM Maintenance

Scripts:

```
build_lhm_from_source.py  
normalize_lhm_semantic_paths.py  
normalize_lhm_class_element.py  
check_lhm_class_element.py  
audit_en16931_coverage.py  
extend_en16931_lhm_coverage.py  
order_lhm_by_en16931_table2.py  
update_lhm_definitions_from_pdf.py  
update_lhm_syntax_sequence_from_ubl_xsd.py
```

Typical checks after LHM edits:

```
& $python .\tests\test_lhm_semantic_paths.py  
& $python .\tests\test_lhm_hierarchical_csv_layout.py  
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py  
& $python .\tests\validate_taxonomy.py
```

Binding Maintenance

Syntax binding CSVs live under:

```
specs/bindings/syntax/
```

Operational scripts:

```
src/syntax_binding.py  
src/syntax_binding_ads_xbrl_gl.py
```

After syntax binding changes:

```
& $python .\tests\test_syntax_binding.py  
& $python .\tests\test_syntax_binding_reverse.py  
& $python .\tests\test_ads_supplier_listing_xbrl_gl.py  
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

When an ADS XBRL GL binding changes, run its target-specific test and validate representative generated instances with Arelle. Supplier Listing changes must verify that the **identifierType=V** selector and its **identifierAddress** children remain in the same vendor identifier reference.

Semantic binding CSVs live under:

```
specs/bindings/semantic/
```

Operational script:

```
src/semantic_binding.py
```

Semantic binding tables start from a target table definition and add **semantic_path**, **type**, and **multiplicity**. The converter derives source columns from **semantic_path** and row scope from **type=C** rows in the binding table.

After semantic binding changes:

```
& $python .\tests\test_ads_invoices_received_psv.py  
& $python .\tests\test_ads_invoices_generated_psv.py  
& $python .\tests\test_semantic_binding_csv_format.py  
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

Round-Trip Artifact Regeneration

When LHM, syntax binding, or taxonomy metadata behavior changes:

```
& $python .\tools\build_roundtrip_test_artifacts.py  
& $python .\tests\test_roundtrip_artifacts.py
```

Artifacts show:

```
source XML  
Structured CSV  
xBRL-CSV metadata JSON  
regenerated UBL XML
```

Continuing Development Workflow

Before editing:

```
git status --short
```

After editing runtime scripts:

```
& $python -m py_compile .\src\syntax_binding.py  
& $python .\tests\test_syntax_binding.py  
& $python .\tests\test_syntax_binding_reverse.py
```

After editing LHM or binding CSV files:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py  
& $python .\tools\build_roundtrip_test_artifacts.py  
& $python .\tests\test_roundtrip_artifacts.py
```

Before sharing changes, run a broad local check and report clearly if Arelle or the UBL schema cache is not available.

psv_viewer.html

tools/psv_viewer.html displays generated ADS PSV files as a browser table. It runs entirely in the browser and does not require a local web server. It supports pipe, comma, and tab delimiters, row filtering, sticky headers, horizontal scrolling, and automatic hiding of columns that are empty in every data row.

13. Machine-Readable Specification Files

Specification Files Guide

This guide explains the specification files under **specs/** and how those files are defined and maintained.

The **specs/** directory contains machine-readable CSV specifications used by the UADC PoC conversion pipeline. These files are inputs to scripts under **src/** and **tools/**. They should remain small, reviewable, and deterministic.

Directory Map

```
specs/  
  Currency.csv  
  CountryCurrency.csv  
  lhm/  
  bindings/  
  XBRL-GL/
```

Currency Tables

Currency.csv maps ISO 4217 currency codes to minor units. It is used when XBRL GL monetary facts need a **decimals** value.

Typical columns:

```
currency_code,minor_unit,currency_name
```

Examples:

```
JPY,0,Japanese Yen  
EUR,2,Euro
```

CountryCurrency.csv maps country examples to currency codes. It is used as reference data for country-oriented examples such as Finland and Estonia.

Examples:

```
FI,Finland,EUR,2,Euro  
EE,Estonia,EUR,2,Euro
```

Runtime XBRL units still use the ISO 4217 currency code from **Currency.csv**.

LHM Specification Files

The LHM files are under:

```
specs/lhm/
```

Important files:

- **EN16931_CIOUS_Invoice_LHM.csv**: generated EN 16931 invoice LHM consumed by Structured CSV conversion and taxonomy generation.
- **source/EN16931_CIOUS_Invoice_LHM_Source.csv**: editable source CSV used to regenerate or adjust the generated LHM.
- **EN16931_CIOUS_Invoice.xlsx**: local reviewer workbook. It is ignored by Git and should not be updated by automation.

The LHM defines:

- semantic paths;
- class and element names;
- multiplicity;
- effective **lhm_level**;
- UBL XPath binding references;
- fields needed for Structured CSV and xBRL-CSV taxonomy generation.

LHM Source CSV

Use **specs/lhm/source/EN16931_CIOUS_Invoice_LHM_Source.csv** for individual adjustments to PDF-derived EN 16931-1 Table 2 items.

Generation flow:

```
python UADC_PoC/tools/build_lhm_from_source.py build `
  UADC_PoC/specs/lhm/source/EN16931_CIOUS_Invoice_LHM_Source.csv `
  UADC_PoC/specs/lhm/EN16931_CIOUS_Invoice_LHM.csv
```

Optional override columns:

- **semantic_path_override**;
- **class_term_override**;
- **element_override**.

When **element_override** is blank, the generator creates a unique UpperCamelCase element name from the semantic path. If final path segments duplicate, it uses the shortest unique semantic path

suffix.

Binding Specification Files

Binding files are under:

```
specs/bindings/
```

Binding authoring and conversion rules are documented in:

- **03_PHASE1_UBL_SYNTAX_BINDING.md** for syntax bindings;
- **04_PHASE2_ADS_PSV_SEMANTIC_BINDING.md** for semantic bindings;
- **02_STRUCTURED_CSV_LHM_BINDINGS.md** for the shared row-scope and function-level processing model.

Binding files connect:

- source syntax to the UADC semantic model;
- UADC Structured CSV to downstream target syntax;
- UADC Structured CSV to flat semantic target tables.

XBRL GL Specification Files

The XBRL GL definition table is under:

```
specs/XBRL-GL/
```

Important file:

- **xbrl-gl.csv**: XBRL GL definition table aligned with **XBRL-GL-PWD-2016-12-01/gl/plt/case-c-b-m-u-e-t-s/gl-plt-all-2016-12-01.xsd** and the imported **gl-cor**, **gl-bus**, **gl-muc**, **gl-ehm**, **gl-taf**, **gl-srcd**, and **gl-usk** modules.

The table preserves existing English and Japanese labels where available. The sequence, module names, cardinalities, type names, and parent-child order are normalized from the selected XBRL GL taxonomy profile.

The **XPath** column records the absolute tuple or fact path from **xbrli:xbrl**. It is generated from the taxonomy parent-child tree so binding tables can point directly to the target XBRL GL location without carrying internal row IDs.

Maintenance Rules

- Keep CSV files UTF-8 or UTF-8 BOM where spreadsheet editing requires it.

- Use structured CSV writers or spreadsheet tools that preserve quoted fields, because descriptions can contain commas.
- Do not commit local reviewer workbook changes unless the workbook is explicitly part of the review deliverable.
- Regenerate derived CSV files from their source CSVs or scripts rather than editing generated files by hand when a generator exists.
- Keep runtime-derived data out of binding tables unless it is deliberately part of the binding contract.

14. Repository Files and Evidence

Repository Files Guide

This guide explains the repository sample files, expected files, references, and figure assets.

Samples

Committed sample files are under:

```
samples/
```

Subdirectories:

- **input/**: source XML samples used by conversion tests.
- **expected/**: stable expected outputs used by focused regression checks.

Important input files:

- **samples/input/openpeppol_ubl_invoice_minimal.xml**: minimal UBL Invoice sample used for the baseline OpenPeppol and EN 16931 conversion check.
- **samples/input/bis-billing3-examples/**: selected BIS Billing 3 invoice examples used to test broader syntax binding coverage.

BIS Billing 3 examples exercise allowances, VAT categories, negative correction invoices, sales order references, and tax accounting currency handling.

Generated Structured CSV, metadata JSON, and regenerated XML are kept in **tests/roundtrip/** or **out/**, not under **samples/**.

References

Lightweight reference material is under:

```
references/
```

Files and subdirectories:

- **peppol_sources.md**: notes and links for OpenPeppol and BIS Billing source material.
- **figures/**: images extracted or prepared from proposal material for Markdown documentation.

Checked source pages:

- Peppol BIS Billing 3.0 upcoming: <https://docs.peppol.eu/poacc/billing/3.0/upcoming/>
- UBL Invoice syntax tree: <https://docs.peppol.eu/poacc/billing/3.0/upcoming/syntax/ubl-invoice/tree/>
- EN 16931 model bound to UBL rules:
<https://docs.peppol.eu/poacc/billing/3.0/upcoming/rules/ubl-tc434/>

The upcoming page identifies the May 2026 Release and separates syntax documentation for UBL Invoice from rules for the EN 16931 model bound to UBL.

Local EN 16931 references used for the base semantic model include **NEN-EN_16931-1_2017+A1_2019_en.pdf**. Clause 6.2 defines how each information element and group is described. Clause 6.3 Table 2 is the semantic data model of the core elements of an electronic invoice.

The current LHM extension includes EN 16931 groups for document-level allowances, document-level charges, document totals, VAT breakdown, invoice line, invoice line period, invoice line allowances, and invoice line charges.

OpenPeppol BIS Billing is treated as a CIUS and syntax-binding source layered on top of the EN 16931 semantic model, not as the primary source of the LHM.

Figures

Figure assets are under:

`references/figures/`

Important file:

- **uadc_poc_processing_flow_figure1.png**: Figure 1 from the PoC proposal, showing the two-stage UADC processing flow from OpenPeppol invoice XML to hierarchical tidy data and downstream audit views.

When adding figures, prefer stable PNG files with descriptive names. Keep intermediate render files only when they are useful for traceability.

15. Complete Test and Round-Trip Procedure

Testing And Round-Trip Artifacts Guide

This guide explains test execution and round-trip review artifacts.

Run commands from the **UADC_PoC** directory. Test scripts are plain Python scripts and can be executed directly with **\$python**. Some can also be run through **pytest** when pytest is installed.

Supporting Files

- **tests/roundtrip/**: reviewable round-trip artifacts. Each case keeps source XML, Structured CSV, xBRL-CSV metadata JSON, and regenerated XML together.
- **tools/build_roundtrip_test_artifacts.py**: rebuilds round-trip artifact sets from sample XML inputs.

Phase 1 Syntax Binding Tests

These tests verify UBL Invoice XML to Structured CSV, xBRL-CSV metadata, and UBL round-trip behavior.

```
& $python .\tests\test_openpeppol_invoice_conversion.py
& $python .\tests\test_lhm_hierarchical_csv_layout.py
& $python .\tests\test_syntax_binding.py
& $python .\tests\test_syntax_binding_reverse.py
& $python .\tests\test_ubl_schema_child_order.py
& $python .\tests\test_bis_billing3_examples_conversion.py
& $python .\tests\test_roundtrip_artifacts.py
```

The forward Phase 1 conversion uses XML parent context recursion, so nested repeated classes preserve parent dimension values such as **dInvoiceLine** with child dimensions such as **dItemAttributes**.

The reverse conversion can derive UBL child element order from XSD files by using **--ubl-schema-root** or **--ubl-schema-url**. **test_ubl_schema_child_order.py** checks this XSD-derived ordering logic without downloading the full UBL package.

test_syntax_binding_reverse.py also checks cross-scope absolute XPath handling: BT-90 must be written below the root **AccountingSupplierParty**, and **PaymentMeans/Invoice** must not exist. **test_bis_billing3_examples_conversion.py** checks the currency-filtered totals in **Allowance-example.xml**: BT-110 is **1225.00** and BT-111 is **9324.00**.

Phase 2 ADS XBRL GL Tests

These tests regenerate Phase 1 Structured CSV first, then use ADS XBRL GL syntax binding CSV files under **specs/bindings/syntax/**.

```
& $python .\tests\test_ads_invoices_received_xbrl_gl.py
& $python .\tests\test_ads_invoices_generated_xbrl_gl.py
& $python .\tests\test_ads_invoices_received_lines_xbrl_gl.py
```

```
& $python .\tests\test_ads_invoices_generated_lines_xbrl_gl.py  
& $python .\tests\test_ads_supplier_listing_xbrl_gl.py  
& $python .\tests\test_ads_customer_master_xbrl_gl.py
```

Expected output:

```
out/phase2/ADS_XBRL_GL/<structured-csv-stem>/<target-view>.xbrl
```

Target view files:

```
Invoices_Received.xbrl  
Invoices_Generated.xbrl  
Invoices_Received_Lines.xbrl  
Invoices_Generated_Lines.xbrl  
Supplier_Listing.xbrl  
Customer_Master.xbrl
```

test_ads_supplier_listing_xbrl_gl.py verifies that Seller name and identifier are written to the **identifierType=V** identifier reference and that Seller Street, City, Country, and Postal Code are written below its **gl-bus:identifierAddress** tuple.

Phase 2 ADS PSV And CSV Tests

These tests use semantic binding files under **specs/bindings/semantic/**.

```
& $python .\tests\test_ads_invoices_received_psv.py  
& $python .\tests\test_ads_invoices_generated_psv.py  
& $python .\tests\test_semantic_binding_indexed_paths.py  
& $python .\tests\test_semantic_binding_csv_format.py  
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

Expected output:

```
out/phase2/ADS_PSV/<structured-csv-stem>/<target-view>.psv
```

Taxonomy And LHM Checks

```
& $python .\tests\test_lhm_semantic_paths.py  
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

```
& $python .\tests\validate_taxonomy.py
```

Round-Trip Artifacts

Round-trip flow:

```
source XML -> Structured CSV -> regenerated XML
```

Artifacts are under:

```
tests/roundtrip/<dataset>/  
  source_xml/  
  structured_csv/  
  metadata_json/  
  roundtrip_xml/
```

Current datasets:

```
tests/roundtrip/openpeppol-minimal/  
tests/roundtrip/bis-billing3-examples/
```

Rebuild:

```
$python = 'python'  
& $python .\tools\build_roundtrip_test_artifacts.py
```

Verify:

```
& $python .\tests\test_roundtrip_artifacts.py
```

The test checks source XML, Structured CSV, metadata JSON, and round-trip XML correspondence. It also checks representative values such as **InvoiceNumber**, **DocumentCurrencyCode**, **InvoiceLineIdentifier**, amount **currencyID**, taxonomy entry points, and xBRL-CSV column concept mappings.

Optional Validation

Arelle metadata validation:

```
& $python .\tests\test_xbrl_csv_metadata_arelle.py
```

UBL 2.1 schema validation:

```
& $python .\tests\test_roundtrip_xml_ubl_schema.py
```

The regenerated XML is not expected to be byte-for-byte identical to the source XML. It is reconstructed from bound CSV values and may differ in XML declaration formatting, namespace placement, indentation, and unbound XML content.

Current Test Execution Report

Last recorded report date:

2026-07-14

Recorded result:

PASS

The complete set of 22 **tests/test_*.py** regression scripts was executed on 2026-07-14 after moving the two simplified converter implementations to **tools/tutorial/**. All 22 completed successfully with no failures. All 14 Python programs under **tools/** also passed **py_compile**.

Scope included EN 16931 LHM-driven syntax binding conversion, Structured CSV generation, xBRL-CSV metadata generation, Arelle validation, UBL reverse conversion, BIS Billing 3 example conversion, LHM checks, and local taxonomy generation checks.

The taxonomy/LHM checks, OpenPeppol conversion, all nine BIS Billing 3 conversions, Structured CSV metadata, ten round-trip artifact cases, and Arelle validation of all ten xBRL-CSV metadata files passed. Absolute currency-filtered XPath evaluation was corrected so **Allowance-example.xml** now writes BT-110 as **1225.00** and BT-111 as **9324.00**. All Phase 2 ADS XBRL GL and ADS PSV/CSV outputs were regenerated and their tests passed.

The Supplier Listing XBRL GL binding was then completed with Seller postal address facts under the **identifierType=V** identifier reference. Supplier Listing was regenerated from all ten Phase 1 inputs, and all ten resulting **Supplier_Listing.xbrl** instances passed Arelle validation. The four ISO 21378 ADC invoice bindings were also applied to all ten Phase 1 inputs, generating 40 CSV target files. Detailed ISO field coverage and known source-data gaps are recorded in **04_PHASE2_ADS_PSV_SEMANTIC_BINDING.md**, Chapter 19.

These results complete the planned Phase 1 and Phase 2 PoC baseline. ISO 21378 completion here means that the four planned invoice views, their mappings, outputs, and gap classifications are complete; it does not mean that EN 16931 contains every audit-system field defined by ISO 21378.

The reverse converter keeps absolute binding XPath paths rooted at the UBL document when a semantic child is stored outside its repeated syntax context. This prevents BT-90 from creating a nested **Invoice** below **PaymentMeans**. All ten regenerated round-trip XML files pass the UBL 2.1 Invoice schema validation when the test is run with an environment containing **xmlschema**.

Part B. End-to-End Tutorial

1. Purpose

This tutorial demonstrates the common UADC flow without requiring the reader to construct long commands. The scripts under **src/tutorial/** call the same operational converters used by the full Phase 1 and Phase 2 workflows.

Run all commands from the repository root.

2. Tutorial Flow

```
UBL Invoice XML
-> Phase 1 syntax binding
-> Structured CSV and xBRL-CSV metadata
-> reverse syntax binding
-> regenerated UBL Invoice XML

Structured CSV
-> Phase 2 ADS syntax binding
-> ADS Invoices Received XBRL GL
```

Tutorial output is written under **out/tutorial/**.

3. Check the Environment

Windows PowerShell:

```
$python = 'python'
& $python .\src\tutorial\00_check_environment.py
```

The script reports:

- the Python executable;
- the detected repository root;
- missing operational scripts or definitions;
- whether the generated OIM taxonomy entry point already exists.

Internally, **REQUIRED_PATHS** lists the LHM, UBL and ADS bindings, sample XML, runtime converters, round-trip builder, and taxonomy generator. The script returns status **1** when any required source file is missing.

4. Generate Phase 1 Structured CSV

```
& $python .\src\tutorial\01_convert_sample_to_structured_csv.py
```

Input:

```
samples/input/openpeppol_ubl_invoice_minimal.xml
```

Outputs:

```
out/tutorial/openpeppol_ubl_invoice_minimal.csv  
out/tutorial/openpeppol_ubl_invoice_minimal.json
```

The wrapper first calls **ensure_taxonomy**. If the local taxonomy is missing, it runs the taxonomy generator regression script. It then invokes **src/syntax_binding.py** with the EN 16931 UBL binding, metadata output path, and taxonomy base.

Open the CSV and observe:

- **dlInvoice** on the invoice row;
- separate rows for repeated invoice lines or VAT breakdowns;
- sparse cells outside the class that owns them;
- fact columns named from the LHM element definitions.

Open the JSON and observe the OIM document type, taxonomy entry point, table template, dimensions, concepts, and currency units.

5. Round Trip to UBL XML

```
& $python .\src\tutorial\02_roundtrip_structured_csv_to_xml.py
```

Output:

```
out/tutorial/openpeppol_ubl_invoice_minimal.roundtrip.xml
```

ensure_structured_csv runs the previous tutorial step when necessary. The wrapper invokes **src/syntax_binding.py** with **--reverse** and the same UBL binding.

The regenerated XML is semantic output, not a byte-for-byte copy. Namespace placement, indentation, XML declaration, and unbound content may differ. The important checks are bound values, hierarchy, UBL child order, and schema validity.

6. Generate a Phase 2 ADS XBRL GL View

```
& $python .\src\tutorial\03_generate_ads_xbrl_gl.py
```

Output directory:

```
out/tutorial/xbrl-gl/
```

The wrapper uses:

```
src/syntax_binding_ads_xbrl_gl.py  
specs/bindings/syntax/ADS_Invoices_Received_XBRL_GL_Binding.csv  
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv  
specs/Currency.csv
```

The generated instance contains XBRL contexts and units plus the XBRL GL tuple hierarchy required by the target view.

7. Inspect the Results

7.1 Structured CSV

Use a spreadsheet editor that preserves quoted CSV cells. Confirm that a parent invoice row and repeated child rows do not combine unrelated class facts.

7.2 Metadata JSON

Confirm that the taxonomy entry is:

```
out/taxonomy/plt/en16931-oim-2026-07-05.xsd
```

7.3 Round-trip XML

Run:

```
& $python .\tests\test_syntax_binding_reverse.py  
& $python .\tests\test_roundtrip_xml_ubl_schema.py
```

7.4 XBRL GL

Load the generated instance in Arelle. Confirm tuple hierarchy and facts rather than expecting a presentation view identical to tuple-oriented legacy samples.

7.5 PSV or CSV

For delimiter-separated Phase 2 output, use **tools/psv_viewer.html**. It reads files locally, supports pipe, comma, and tab delimiters, filters rows, keeps headers visible, and hides wholly empty columns.

8. What Happens Internally

The tutorial wrappers use **subprocess.run** with **check=True**. A child converter failure therefore stops the wrapper immediately.

Phase 1 performs these internal steps:

1. load binding class and fact rows;
2. derive dimensions and direct class fields;
3. walk XML class contexts recursively;
4. emit the parent row and repeated child rows;
5. write metadata using the same column and dimension layout.

Reverse conversion groups rows by dimensions, rebuilds XML nodes from absolute binding XPath, applies predicates and attributes, then orders UBL children from schema-derived syntax sequences.

ADS XBRL GL generation validates fact ownership, selects the source rows for the target view, creates contexts and currency units, follows target XPath, and writes facts in XBRL GL schema order.

9. Simplified Tutorial Implementations

The programs under **tools/tutorial/** are different from the wrappers above. They implement smaller converters for learning and binding experiments:

```
tools/tutorial/syntax_binding_sample.py  
tools/tutorial/semantic_binding_sample.py
```

They do not implement the complete hierarchical row ownership, repeated scope, metadata, reverse UBL, or Phase 2 behavior. Use them to understand a small algorithm, not to generate PoC deliverables.

10. Next Steps

After completing the tutorial:

1. Read **02_STRUCTURED_CSV_LHM_BINDINGS.md** for LHM, dimensions, and Structured CSV rules.
2. Read **03_PHASE1_UBL_SYNTAX_BINDING.md** before changing Phase 1 bindings.
3. Read **04_PHASE2_ADS_PSV_SEMANTIC_BINDING.md** before generating ADS PSV, ADS XBRL GL, or ADC.
4. Use **01_ENVIRONMENT_TESTS_TUTORIAL.md** to run the relevant regression and external validations.